1 — Introduction

Machine Learning (MPA-MLR)

Tomáš Vičar
Department of Biomedical Engineering · FEEC, Brno University of Technology

2025

Course Outline

1. Introduction, optimization, overview
2. Model evaluation and pre-processing
3. Feature selection and dimensionality reduction
4. Linear models, kernel methods
5. Bias–variance tradeoff, decision trees, bagging, boosting, random forest
6. Neural networks
7. Deep learning I
8. Deep learning II
9. Deep learning III
10. Deep learning IV
11. Probabilistic models I
12. Probabilistic models II

Course evaluation

The course evaluation rules will be addressed in the practical exercises.

Outline

1. **Definitions** — what is AI, ML, DL and how they relate
2. **Branches of Machine Learning** — supervised, unsupervised, reinforcement, self-supervised
3. **Recapitulation of simplest algorithms** — distances, k-NN, k-means, UPGMA
4. **Optimization** — the language in which almost every ML method is written

1 · Definitions

> What exactly do we mean by artificial intelligence, machine learning and deep learning — and how do they fit into each other?

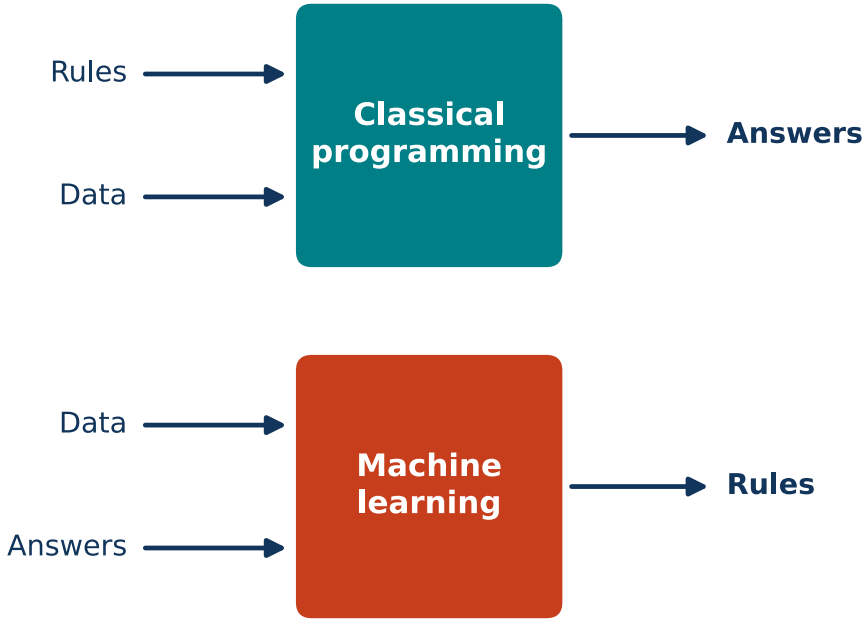
Classical Programming vs. Machine Learning

Classical Programming

- Human provides **rules** (algorithms) + **data**.
- Computer executes rules to produce **answers**.
- Example: sorting numbers, calculating taxes, signal FIR filtering.

Machine Learning

- Human provides **data** + **answers (labels)**.
- Computer learns the **rules** (model) from the data.
- Example: spam filter, image classification, LLM (Large Language Model).



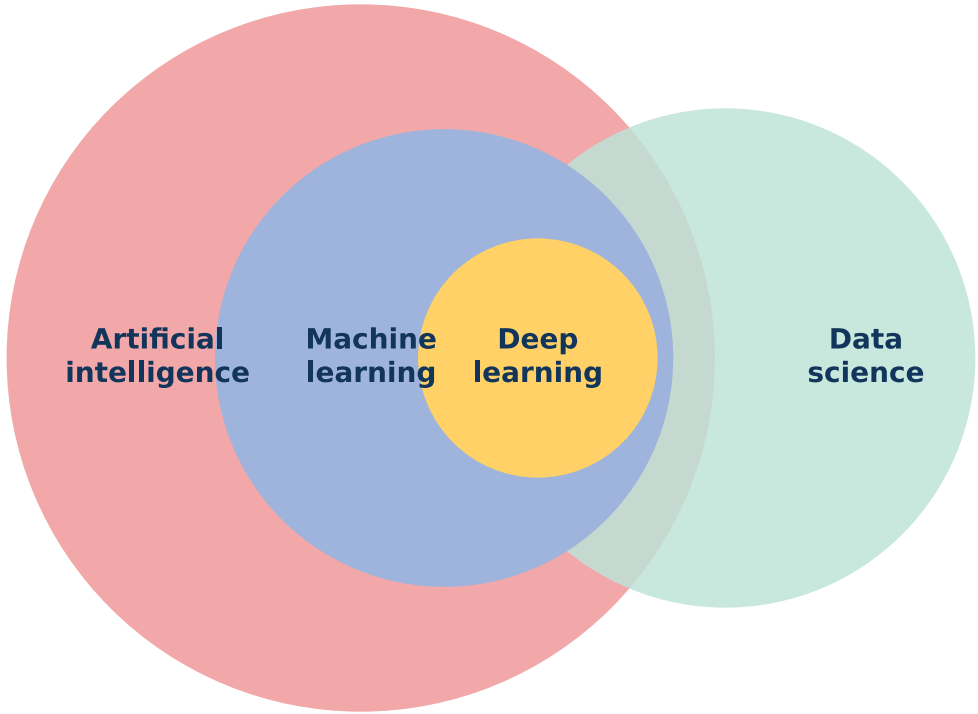
Artificial Intelligence vs. Machine Learning

Artificial Intelligence (AI)

- Broad field of computer science focused on creating systems that perform tasks requiring human intelligence.
- Understanding language, decision-making, autonomous driving.
- **AI = umbrella term**.

Machine Learning (ML)

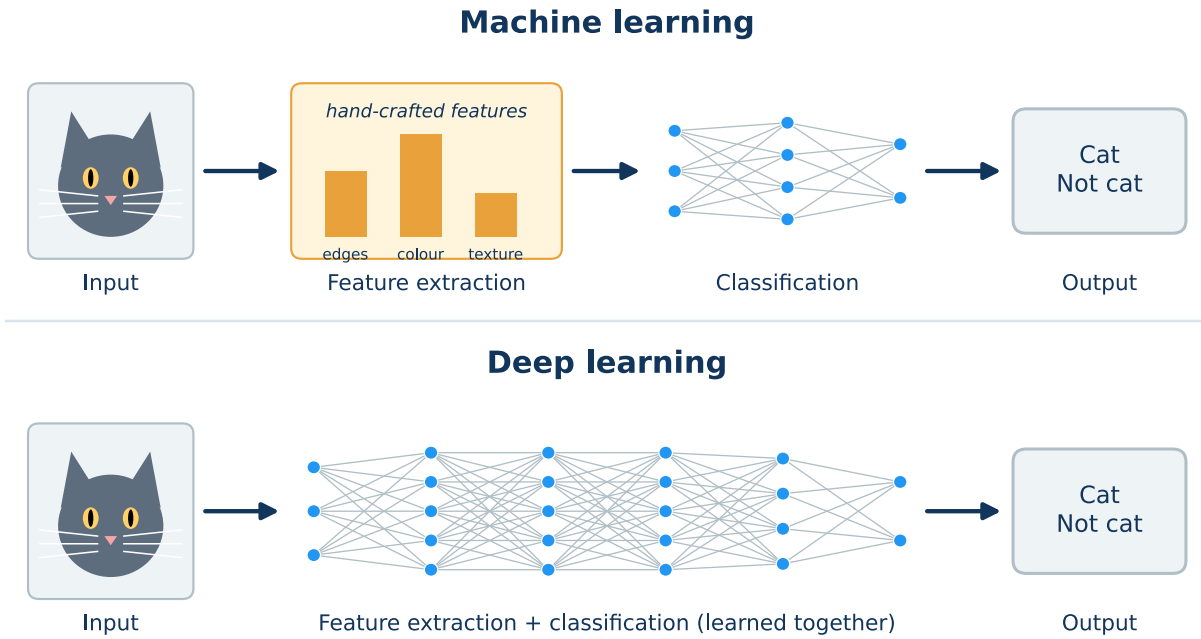
- Subset of AI focused on learning patterns from data.
- Spam detection, recommender systems, medical image analysis.
- **ML = one approach to achieve AI**.



Machine Learning vs. Deep Learning

- **Machine Learning:** algorithms learn patterns from data, often need *manual feature engineering*, work well with smaller datasets.
- **Deep Learning:** subset of ML using *neural networks with many layers*, learns features **automatically**, needs big data + high compute.

Machine Learning	Deep Learning
Manual features	Automatic features
Small data OK	Needs big data
Traditional algorithms	Neural networks

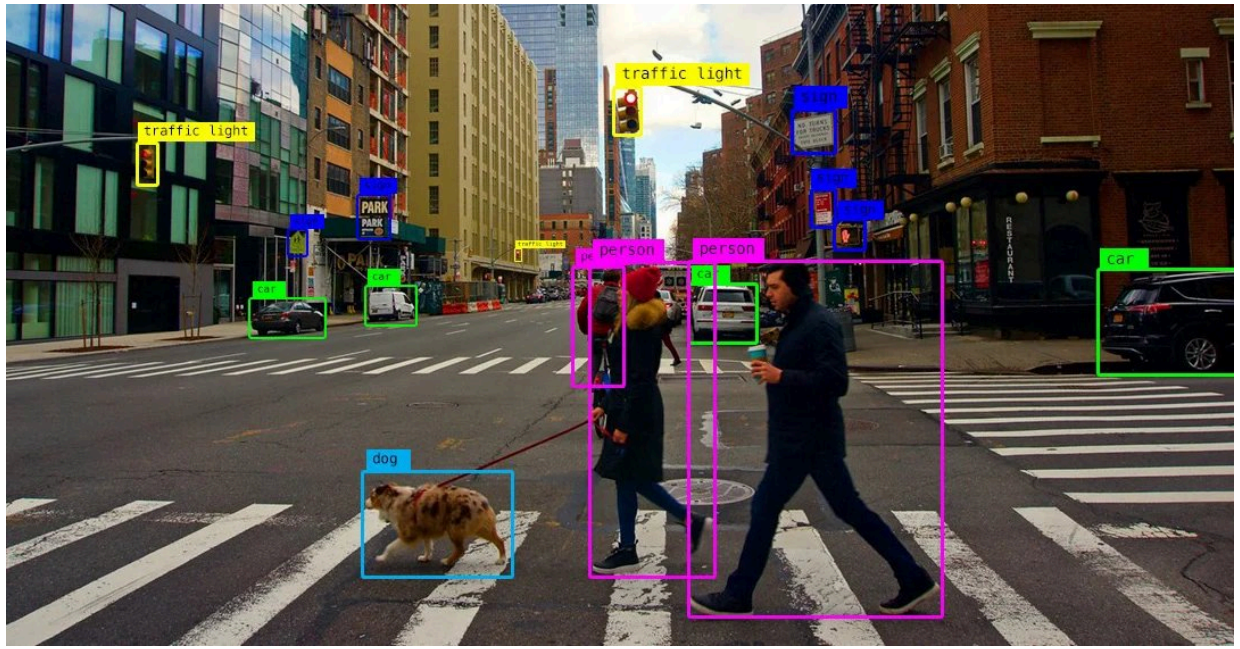


Related fields: Data Science

- Interdisciplinary field that combines *statistics*, *machine learning* and *computer science*.
- Goal: extract **insights and knowledge** from structured and unstructured data.
- Applications: business analytics, healthcare, recommender systems.

Related fields: Computer Vision

- Subfield of Artificial Intelligence (AI).
- Enables computers to *derive meaningful information* from digital images, videos and other visual inputs.
- Systems can **act or make recommendations** based on that information.
- Applications: object detection, medical imaging, autonomous driving.



Related fields: Robotics

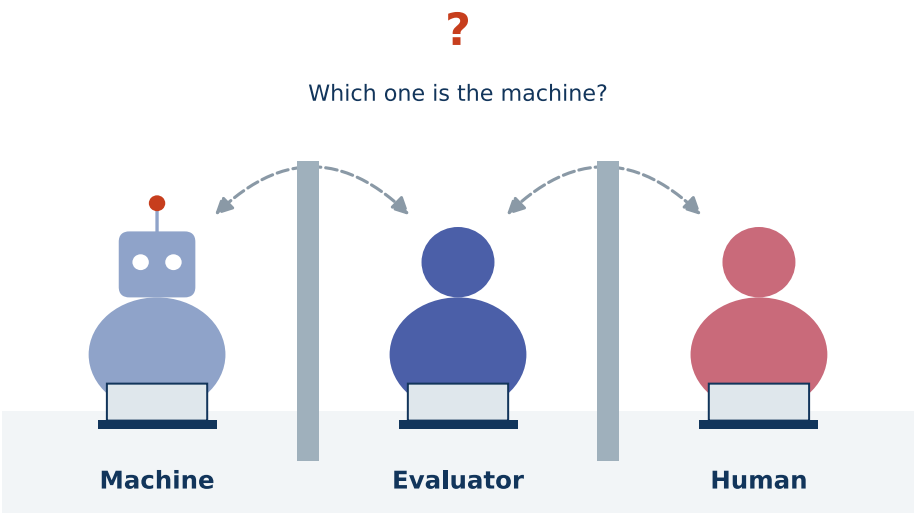
- Field focused on *designing, building and programming robots*.
- Aims to enable robots to operate in **complex, real-world scenarios**.
- Combines AI, computer vision, control theory and mechanics.
- Applications: industrial automation, service robots, autonomous vehicles.

The Turing Test

AI raises the question: does *intelligent behavior* imply or require the existence of a **mind**? Can machines truly “think”, or only *simulate* thinking?

The Turing Test (Alan Turing, 1950)

- A test of a machine’s ability to exhibit *intelligent behavior*.
- If a human evaluator cannot reliably distinguish between responses from a machine and a human, the machine is said to have demonstrated AI.
- LLMs *pass the Turing Test* — does this mean they are intelligent?



Narrow AI vs. General AI

Narrow AI (Weak AI)

- AI systems designed for a *specific task*.
- Very good at what they are trained for, but cannot generalize.
- Examples: spam filters, recommendation systems, image classifiers, voice assistants.

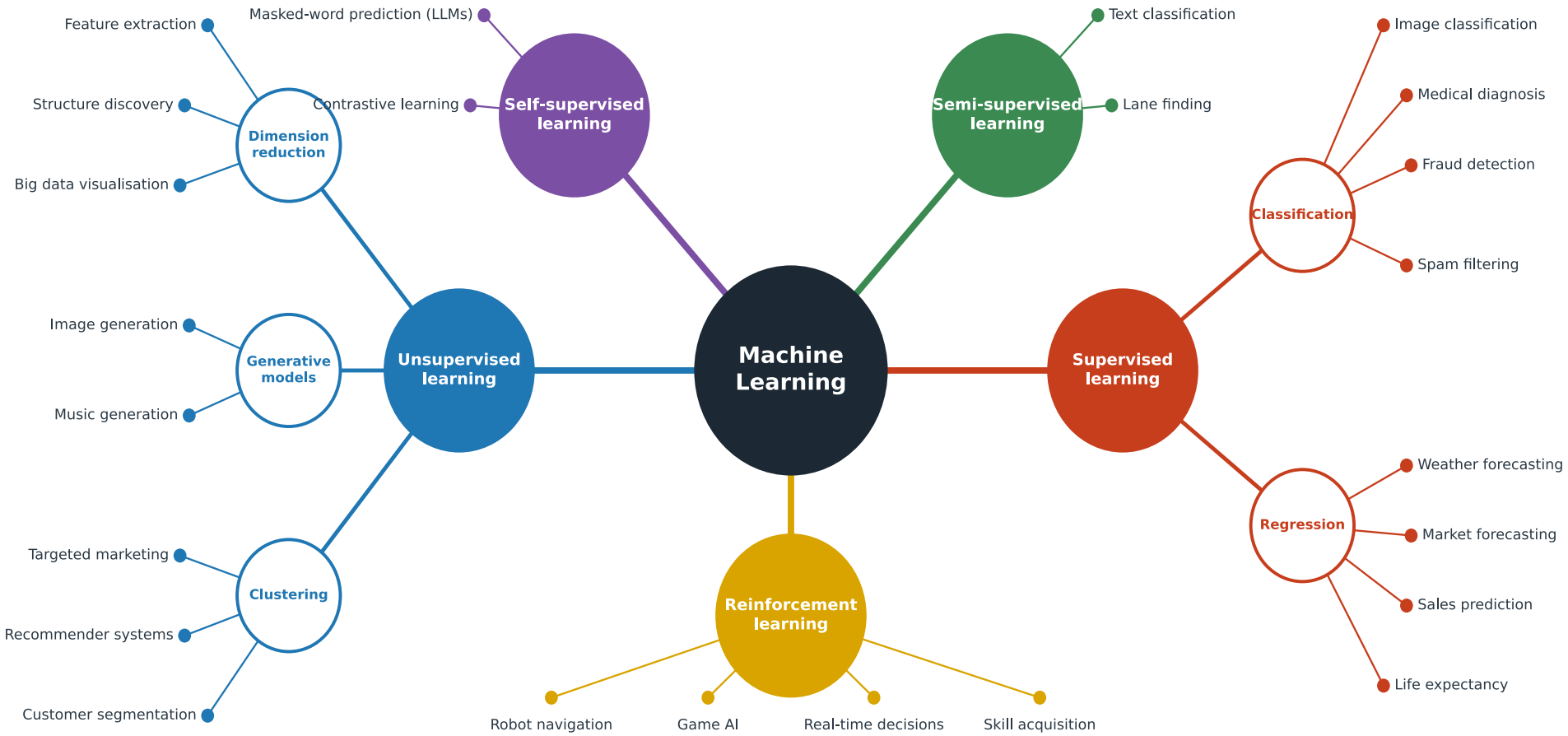
General AI (Strong AI)

- Hypothetical AI with the ability to *understand, learn and apply knowledge across domains*.
- Would perform any intellectual task that a human can do.
- Still a **theoretical concept**.
- AGI — Artificial General Intelligence.

2 · Branches of Machine Learning

What kind of information do we give the algorithm — labels, nothing at all, or a reward?

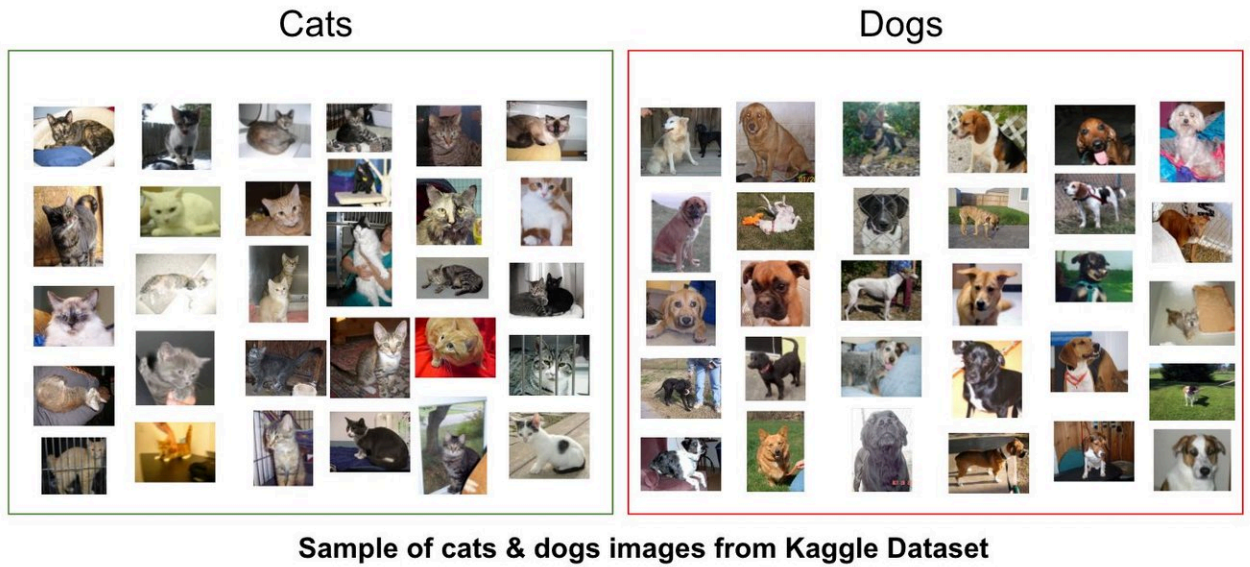
Branches of Machine Learning



Supervised Learning

Definition

- A machine learning paradigm where models learn from *labeled data*.
- Training data consists of input variables $\mathbf{x}$ and known outputs y.
- Goal: learn a mapping $f : \mathbf{x} \mapsto y$ that generalizes to unseen data.
- Example: cat vs. dog recognition.

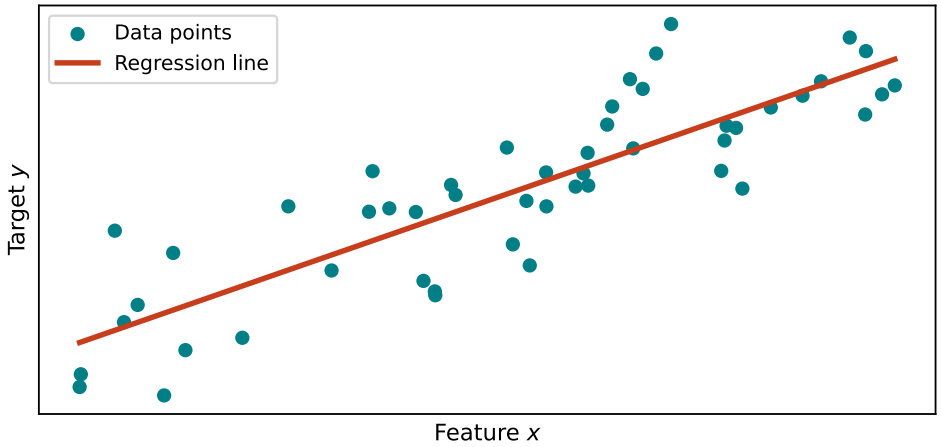


Sample of cats & dogs images from Kaggle Dataset

Regression

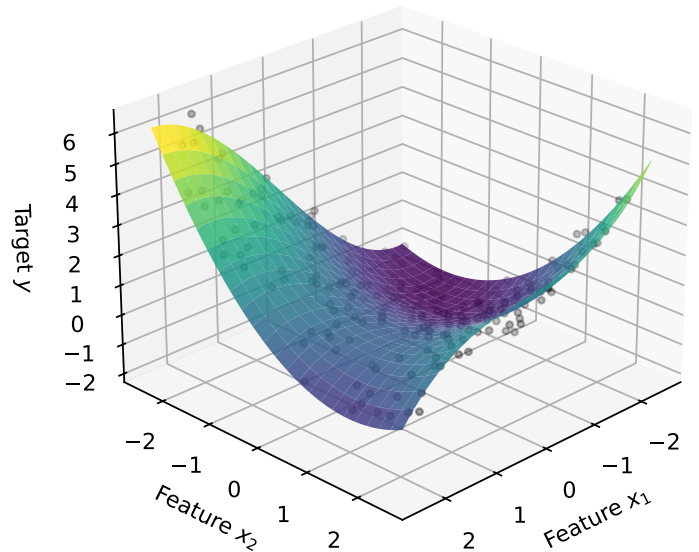
Definition

- Regression is a *supervised learning* technique used to model the relationship between input variables $\mathbf{x}$ (*features*) and a continuous output variable y (*target*).
- The goal is to **predict or estimate** the target value based on new input data.



Basic Concept

- Fits a function $\hat{y} = f(\mathbf{x})$ to minimize the error between predicted $\hat{y}$ and true values.
- Example: predicting house prices, predicting cholesterol level.



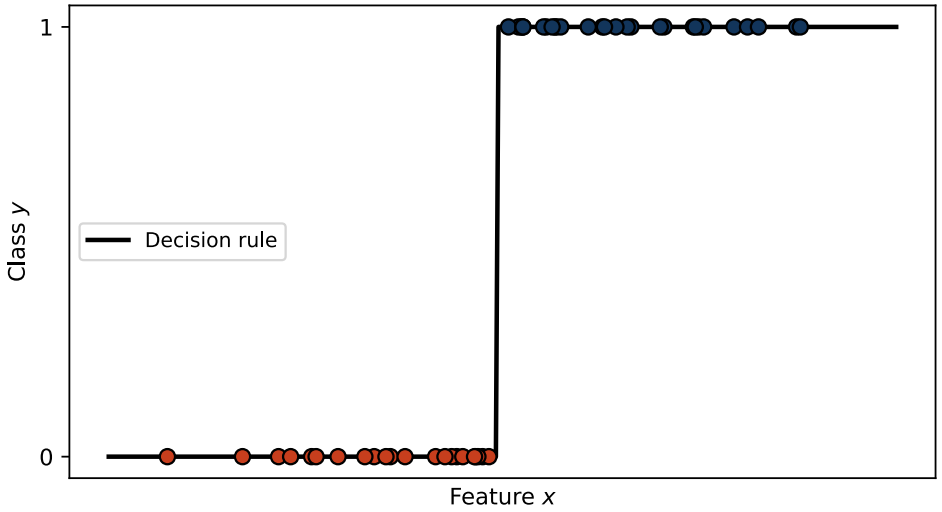
Classification

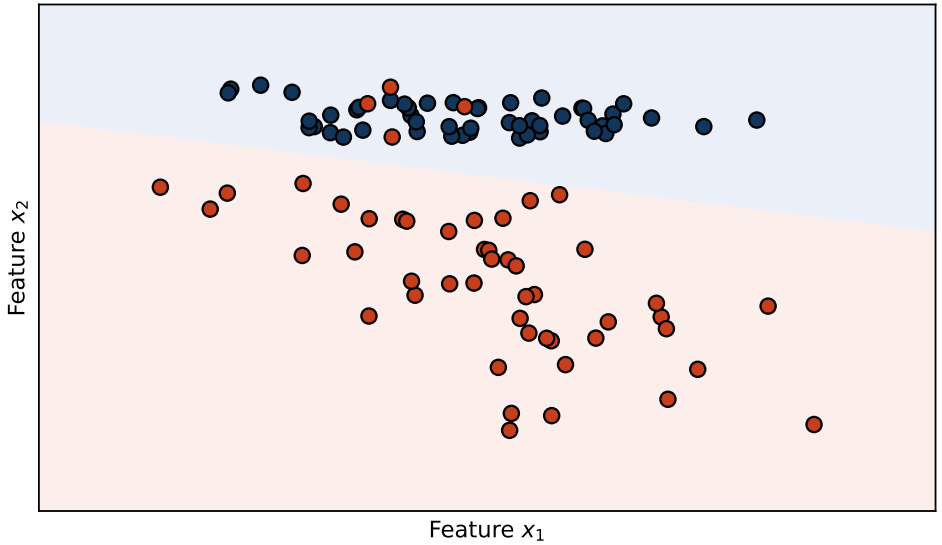
Definition

- Classification is a *supervised learning* technique used to assign input variables $\mathbf{x}$ (*features*) into one of several **discrete categories** (*classes*).
- The goal is to **predict the class label** y for new, unseen data.

Basic Concept

- Learns a decision function $\hat{y} = f(\mathbf{x})$ that separates data into distinct classes.
- Example: predicting **cat vs. dog**, classifying **healthy vs. diseased** patients.





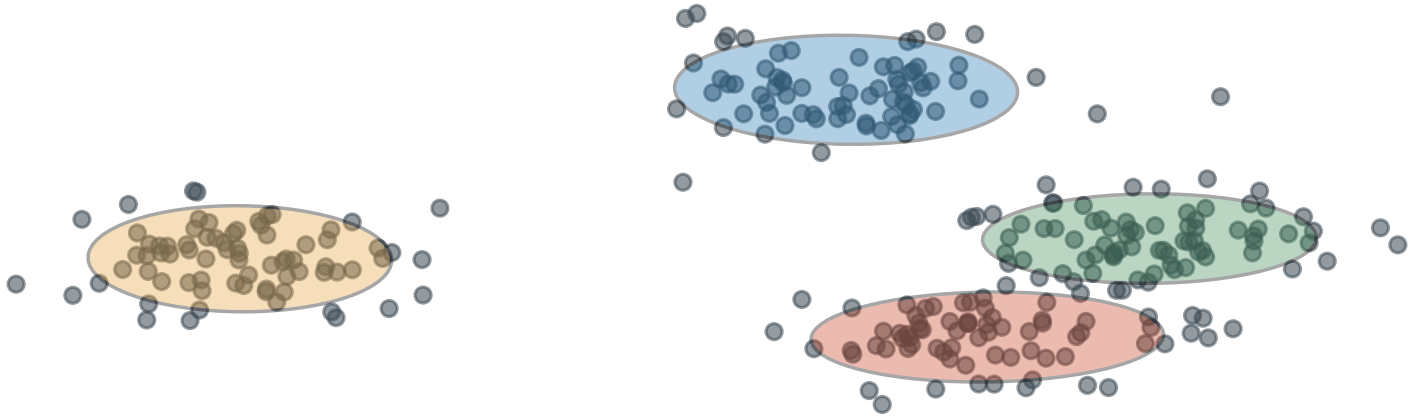
Unsupervised Learning

Definition

- A machine learning paradigm where models learn from *unlabeled data*.
- Only input variables $\mathbf{x}$ are given, without known outputs y.
- Goal: discover **hidden patterns, structures or groupings** in data.

Examples

- **Clustering:** grouping similar data (e.g. customers by shopping behavior).
- **Dimensionality Reduction:** compressing high-dimensional data (e.g. PCA for visualisation).
- **Generative Models:** learning a distribution to create new samples (image generation).



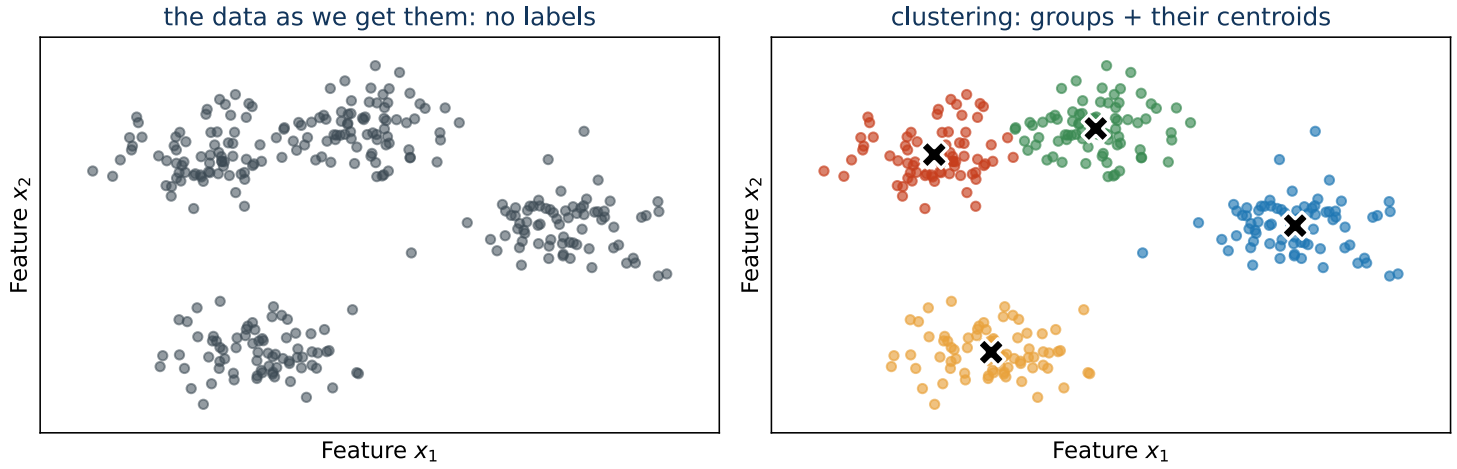
Unsupervised Learning: Clustering

What it is

- Group the samples so that **similar ones end up together**.
- No labels — and **nobody says how many groups** there are or what they mean.
- “Similar” means *close*, so everything hangs on the distance we pick.

Typical uses

- Customer segments from shopping behaviour.
- Cell types from single-cell measurements.
- Grouping images or documents before anyone annotates them.



Two clustering algorithms, **k-means** and **UPGMA**, are recapitulated in the next block of this lecture.

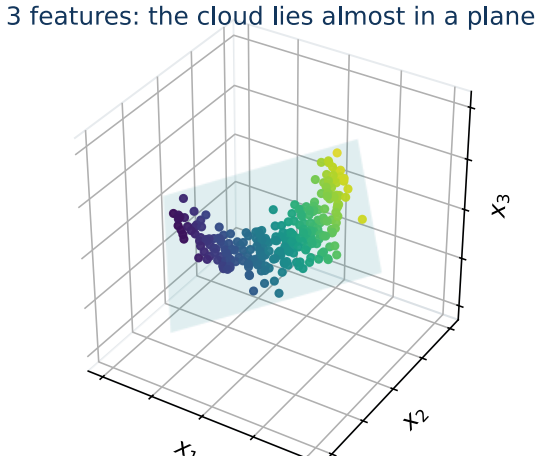
Unsupervised Learning: Dimensionality Reduction

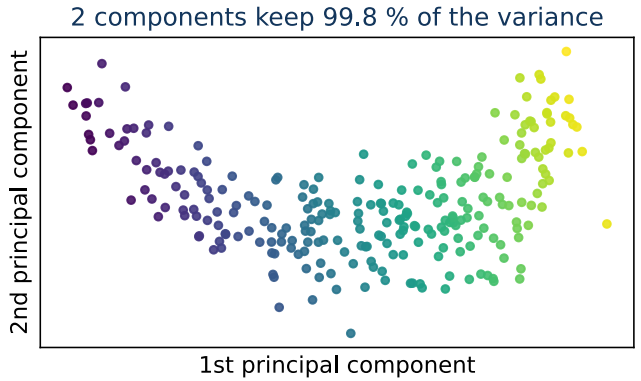
What it is

- Replace many features by a few new ones and keep the **structure** of the data.
- The new axes are combinations of the old ones, ordered by how much variation they carry.
- Dropped are the directions in which the data barely change.

Typical uses

- **Visualisation** — two dimensions instead of two hundred.
- **Denoising** — the dropped directions are mostly noise.
- **Compression before a classifier** — fewer inputs, fewer parameters.





Feature selection, PCA, t-SNE and UMAP are the subject of the whole third lecture.

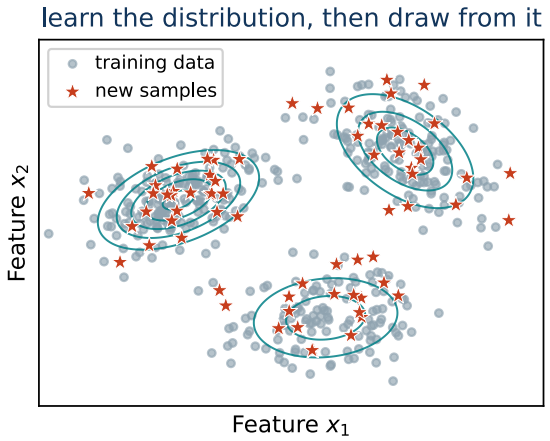
Unsupervised Learning: Generative Models and LLMs

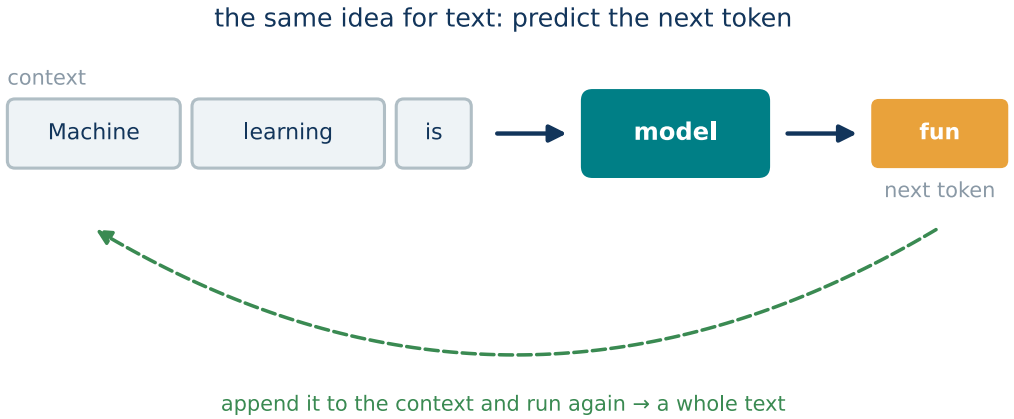
What it is

- The model learns **what the data look like** — their distribution, not a label.
- It can then **draw new samples** — data like the training set, but not in it.
- Raw unlabeled data is enough, and of that there is a lot.

Large language models (LLMs)

- Trained on a huge amount of text to **predict the next token** (a word or a piece of one).
- The target is the next word of the text itself — no annotation: **self-supervised** learning, on a slide of its own in a moment.
- Generating text = predict, append, repeat. Image generators (diffusion models) do the same with pixels.





The transformer behind today's language models is the tenth lecture; generative models come back in the eleventh and twelfth.

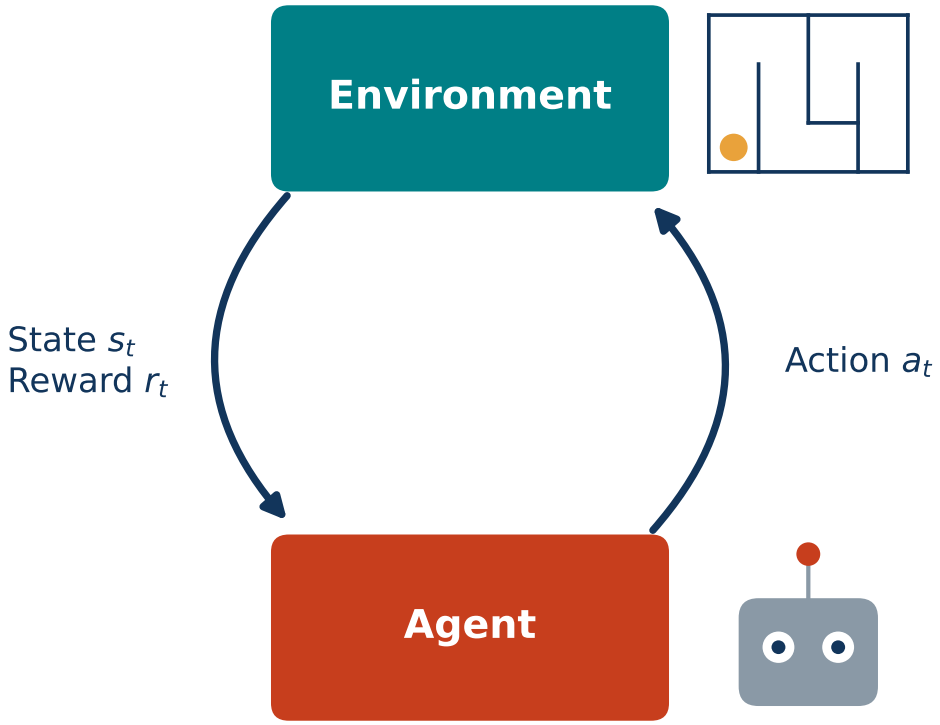
Reinforcement Learning

Definition

- A machine learning paradigm where an **agent** learns to make decisions by interacting with an *environment*.
- The agent observes the **state**, takes an **action** and receives a **reward**.
- Goal: learn a policy that maximizes cumulative reward over time.

Examples

- Training robots to navigate mazes.
- Game playing (e.g. AlphaGo, Atari games).
- Dynamic resource allocation.



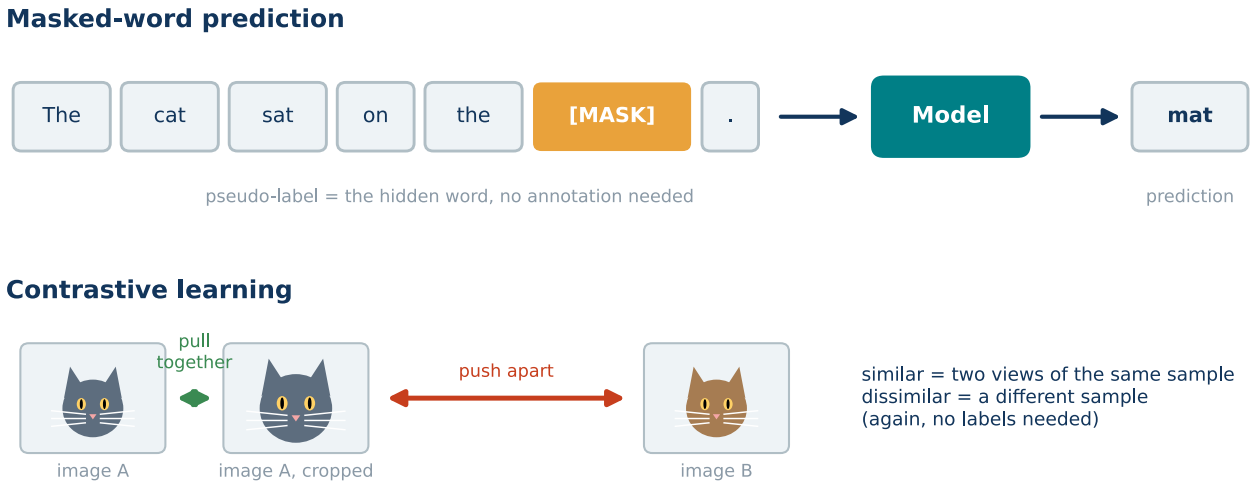
Self-Supervised Learning

Definition

- Learns *representations* from raw unlabeled data by creating **pseudo-labels** from the data itself.
- The training task is often a **pretext task**, solved to enable downstream tasks.

Examples of pretext tasks

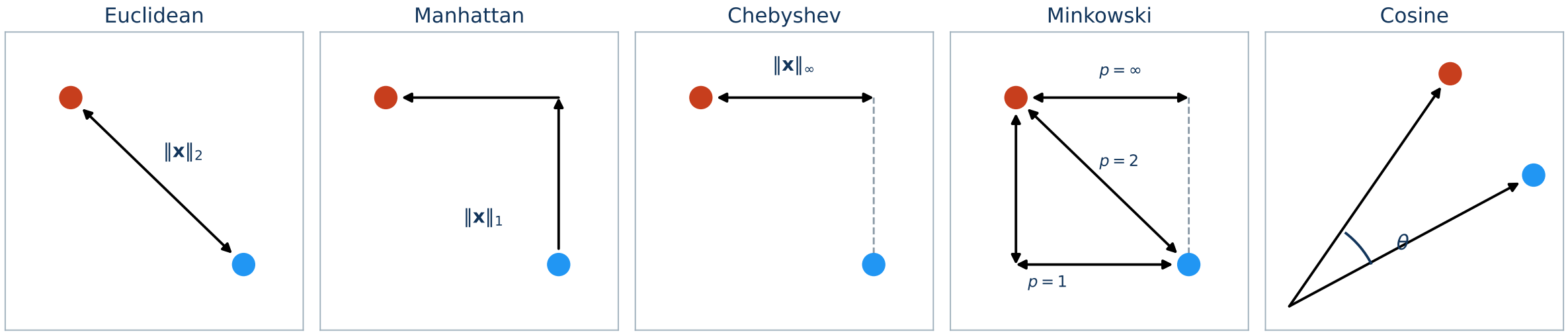
- Predicting missing parts of the input (e.g. masked words in text, missing pixels in images).
- Contrastive learning: bring **similar samples closer**, push **dissimilar apart**.
- Next-frame prediction in videos, *next-word prediction in a sentence (LLMs)*.



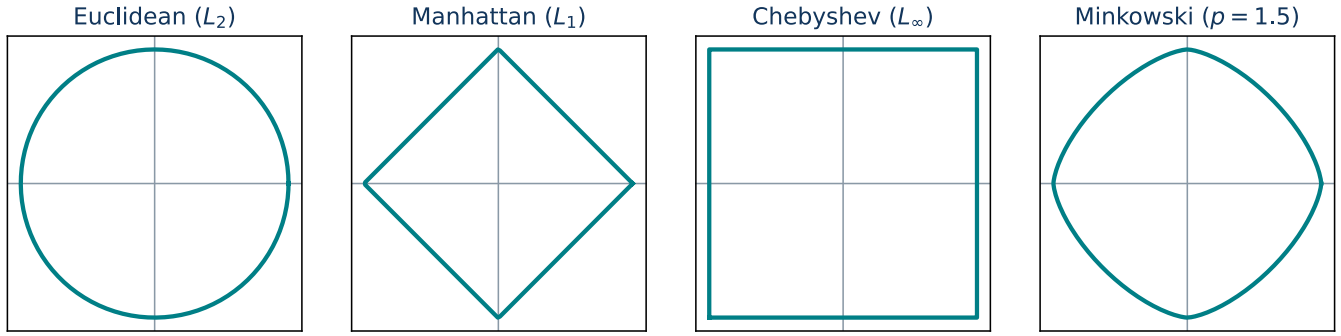
3 · Recapitulation of simplest algorithms (from BPC-UIN)

Distances, nearest neighbours, k-means and UPGMA — a quick refresher from the bachelor course.

Distances and vector norms



Unit balls



$$\|\mathbf{x}\|_2 = \sqrt{\sum_i x_i^2} \quad \|\mathbf{x}\|_1 = \sum_i |x_i|$$

$$\|\mathbf{x}\|_\infty = \max_i |x_i| \quad \|\mathbf{x}\|_p = \left(\sum_i |x_i|^p\right)^{1/p}$$

For a distance, let $\mathbf{x} = \mathbf{a} - \mathbf{b}$ be the difference vector.

Example: $\mathbf{a} = (1, 1), \mathbf{b} = (2, 3)$, so $\mathbf{x} = (-1, -2)$:
$\|\mathbf{x}\|_2 = \sqrt{5} \approx 2.24, \|\mathbf{x}\|_1 = 3, \|\mathbf{x}\|_\infty = 2.$

code example

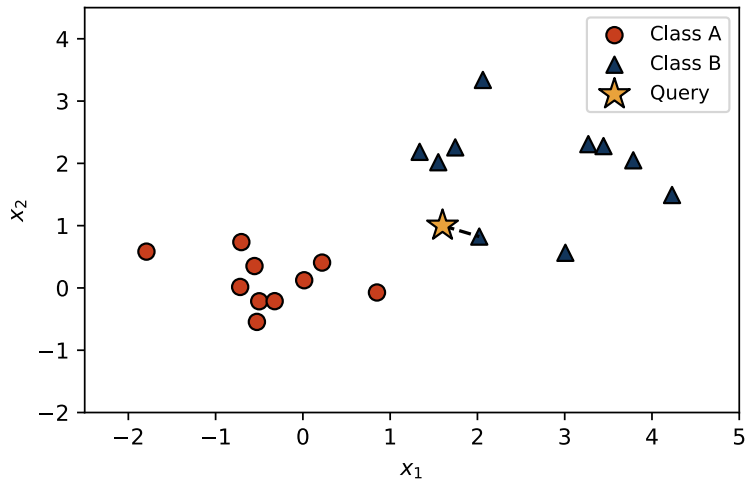
Nearest Neighbor (1-NN)

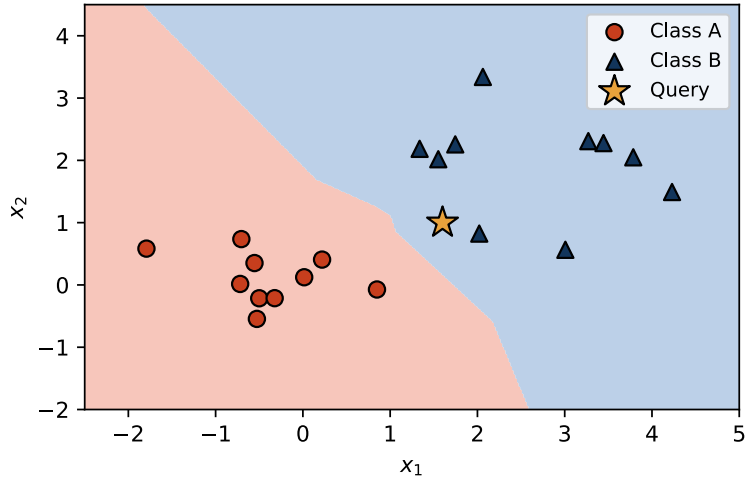
Idea

- For a new sample $\mathbf{x}_*$, find the *closest* training point according to a chosen distance metric (typically Euclidean).
- Predict the same class/value: $\hat{y}(\mathbf{x}_*) = y_{(1)}$.

Properties

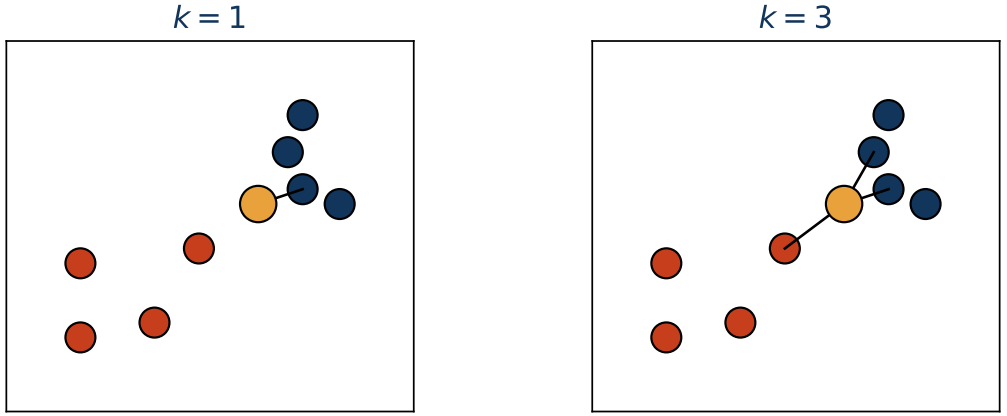
- No explicit training phase (lazy learning).
- Sensitive to noise, scaling of features and outliers.
- Distance search is expensive for large n (use a kd-tree).

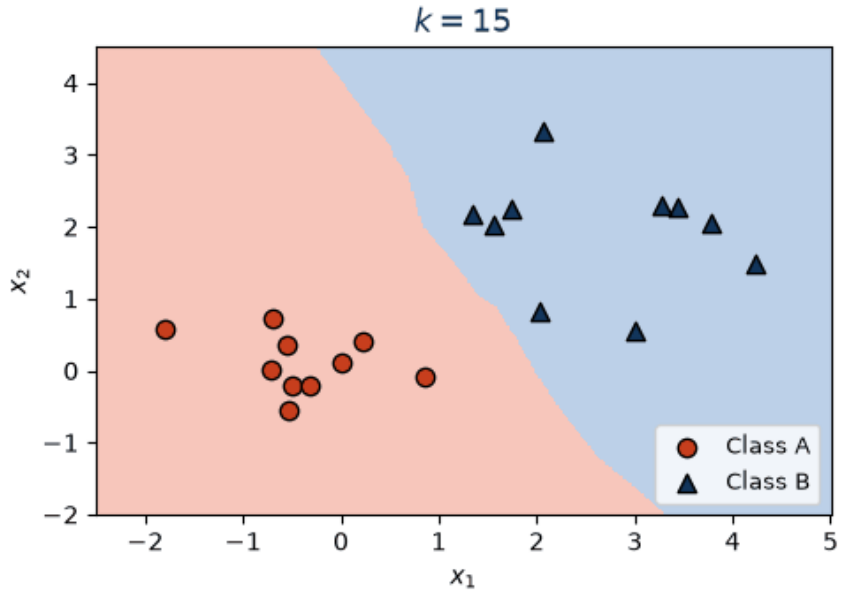




k-Nearest Neighbors (k-NN)

- For $\mathbf{x}_*$, find the k closest neighbors $\mathcal{N}_k(\mathbf{x}_*)$.
- Majority vote for classification, average for regression.
- Choice of k regularizes the model (bias–variance trade-off).
- Simple and powerful classifier.





Playground: http://vision.stanford.edu/teaching/cs231n-demos/knn/ · code example

k-means Clustering

Idea

- Partition n data points into k clusters.
- Each cluster is represented by its centroid (mean).
- Minimize the **inertia** (within-cluster variance):

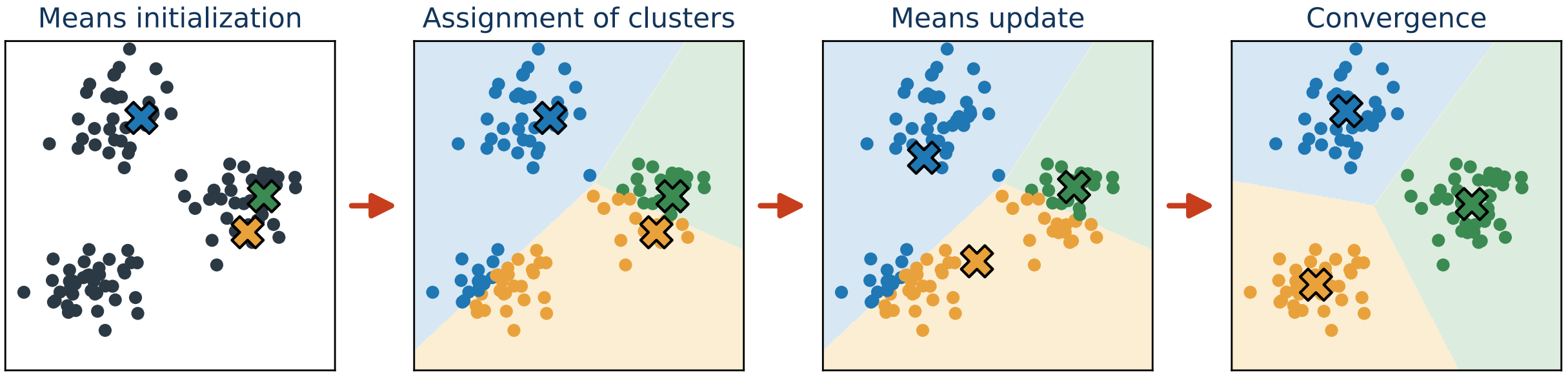
$$J = \sum_{j=1}^{k} \sum_{\mathbf{x}_i \in C_j} \|\mathbf{x}_i - \boldsymbol{\mu}_j\|_2^2$$

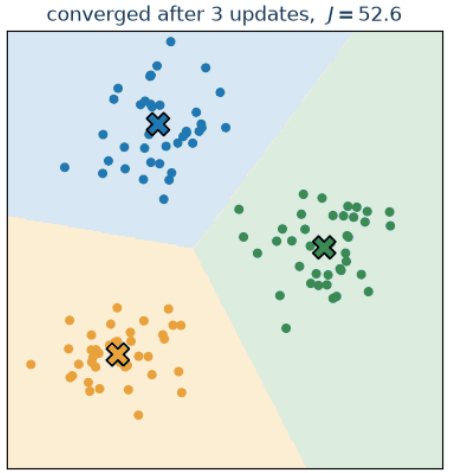
Algorithm (Lloyd's)

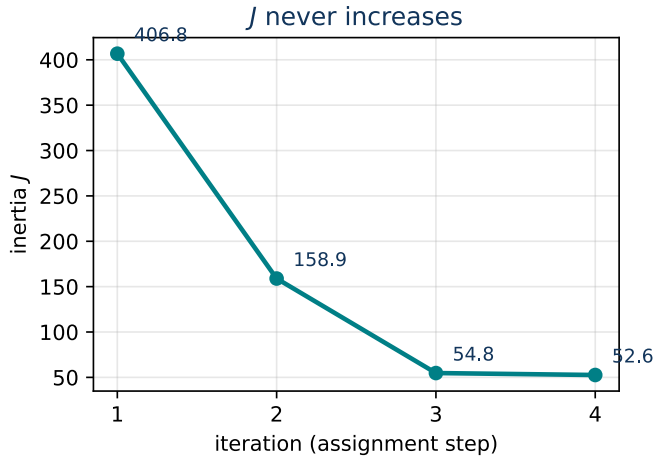
1. Initialize k centroids (random samples).
2. Assign each $\mathbf{x}_i$ to the nearest centroid.
3. Update centroids as means of assigned points.
4. Repeat 2–3 until convergence.

- Sensitive to initialization (use multiple restarts).
- Finds local minima, not guaranteed global optimum.

k-means Clustering: the algorithm step by step







Every assignment step and every means update can only **decrease** J — that is why Lloyd's algorithm always converges (to a local minimum).

code example

UPGMA

Unweighted Pair Group Method With Arithmetic Mean

Idea

- A hierarchical clustering method.
- Iteratively merges the two closest clusters.
- Distance between clusters = average of all pairwise distances.
- Produces a rooted tree (dendrogram).

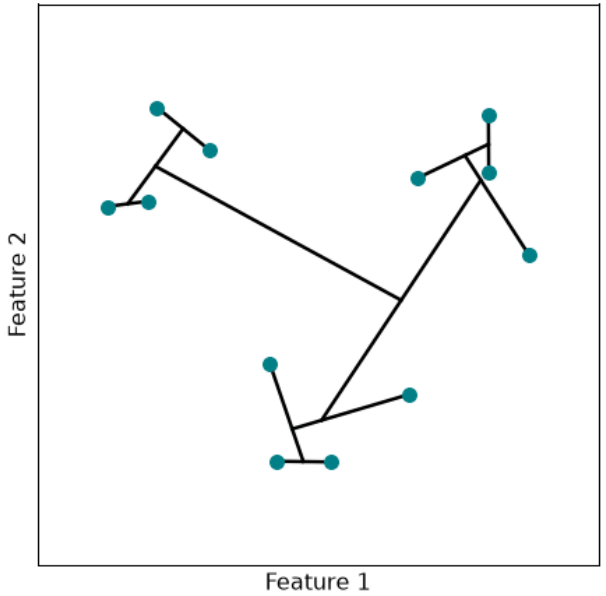
Algorithm

1. Start with each sample as its own cluster.
2. Find the pair of clusters with the smallest distance.
3. Merge them into a new cluster.
4. Update the distance matrix using averages.
5. Repeat until one cluster remains.

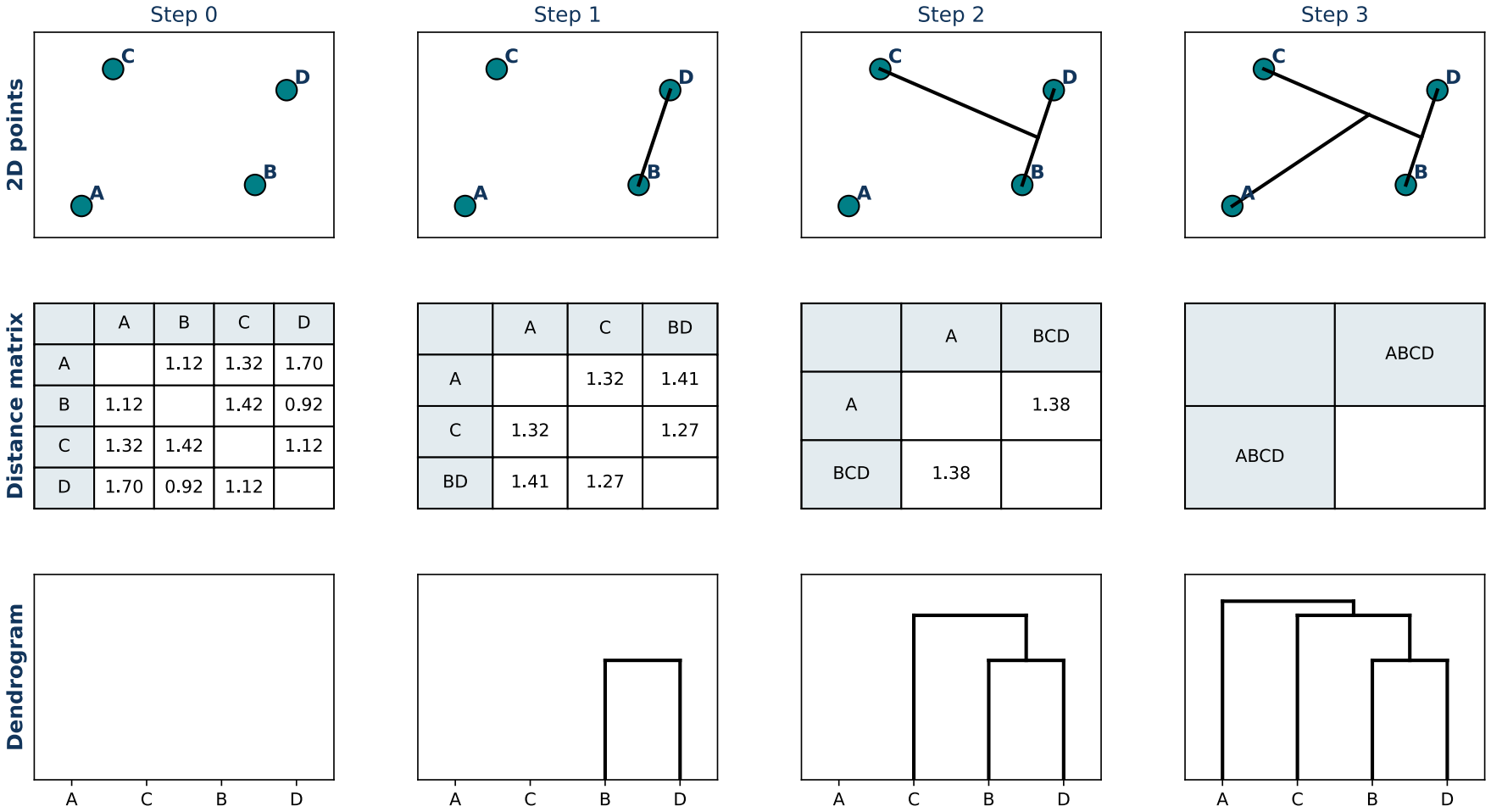
Cluster distance definition

Let C_i and C_j be the clusters to be merged and $|C_i|$ the number of elements in C_i. The distance to another cluster C_k is updated as

$$d(C_{i\cup j}, C_k) = \frac{|C_i|\, d(C_i, C_k) + |C_j|\, d(C_j, C_k)}{|C_i| + |C_j|}$$



UPGMA: practical construction



code example

4 · Optimization

Almost every machine learning method boils down to minimizing something. Let us recall what that means.

Mathematical Background for ML

Key areas of knowledge:

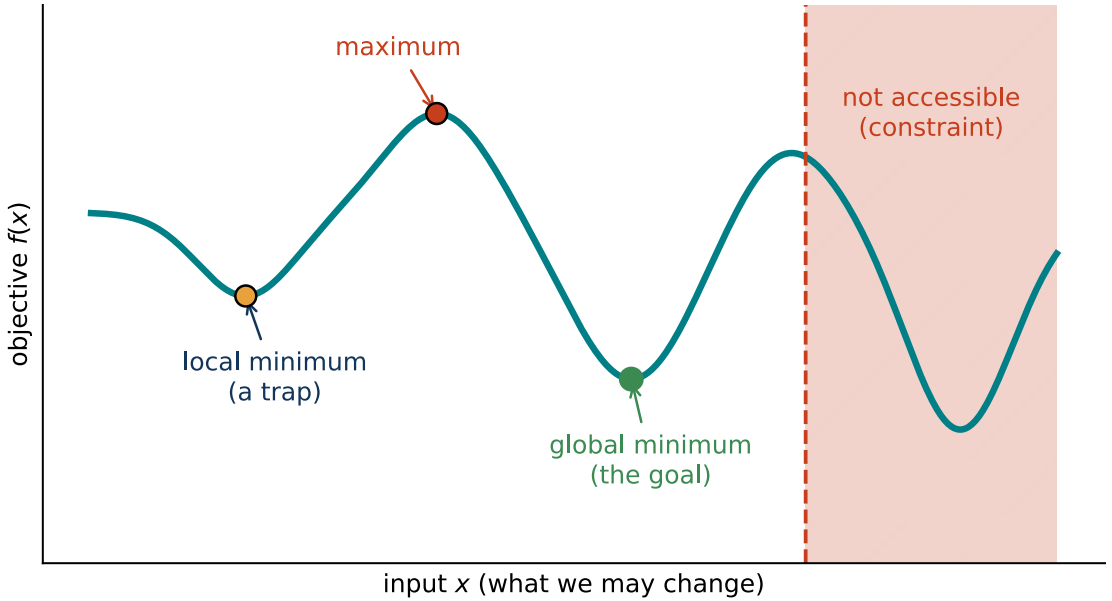
- **Linear Algebra:** vectors, matrices, matrix operations, eigenvalues and eigenvectors — matrices are an efficient way to represent complex calculations.
- **Calculus:** derivatives, gradients, partial derivatives, chain rule, *optimization concepts* — optimization is the key to model training and it often uses derivatives.
- **Probability and Statistics:** random variables, distributions, expectation, variance, Bayes' theorem — they provide tools to describe randomness, estimate patterns from data and decide how likely events are.

Let us recapitulate the key concepts of optimization (more details in MPA-EAL).

Optimization: intuition

- We are changing some inputs to get the best possible output.

- Process of finding the **best solution** among many possible ones.
- Typically means **minimizing a cost** or maximizing a reward.
- Imagine a **landscape**: optimization is searching for the lowest valley (minimum) or the highest peak (maximum).
- Constraints define which parts of the landscape are **accessible**.



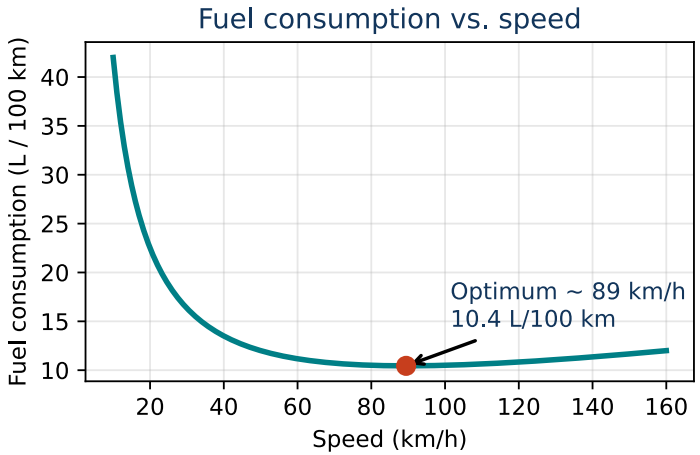
- Almost everything can be defined as an optimization problem!
- Examples: investment portfolio selection — maximize profit; selecting the fastest route — minimize time…
- Maximization can be done by minimizing the negative values.

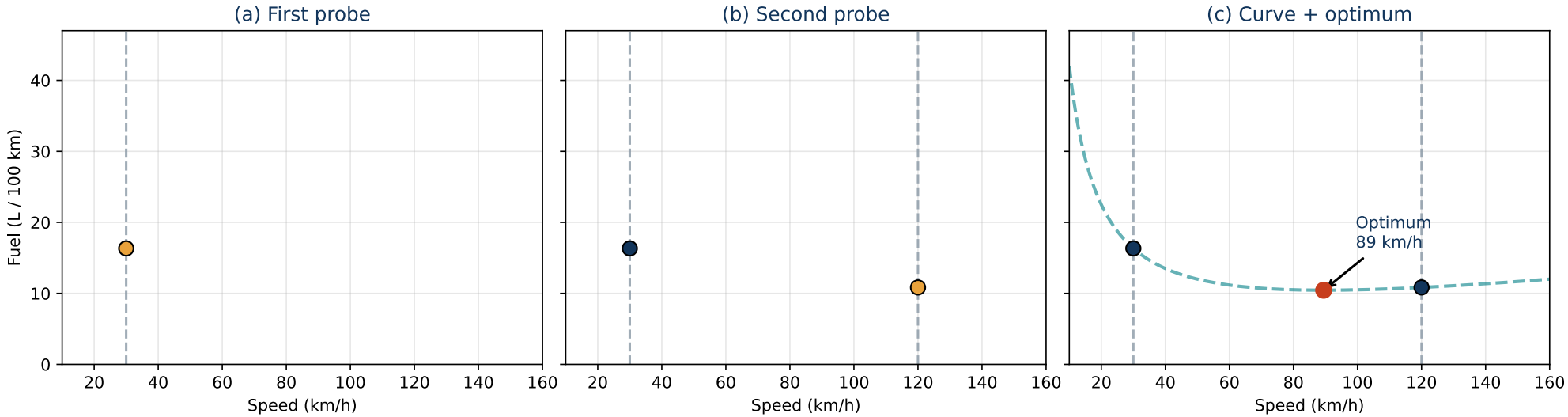
Optimization example 1D: speed vs. fuel consumption

Idea

At very low speed the ride is inefficient (idling losses, short gearing). At very high speed, aerodynamic drag $\propto v^2$ makes fuel consumption rise again. Therefore an **optimal speed** naturally exists.

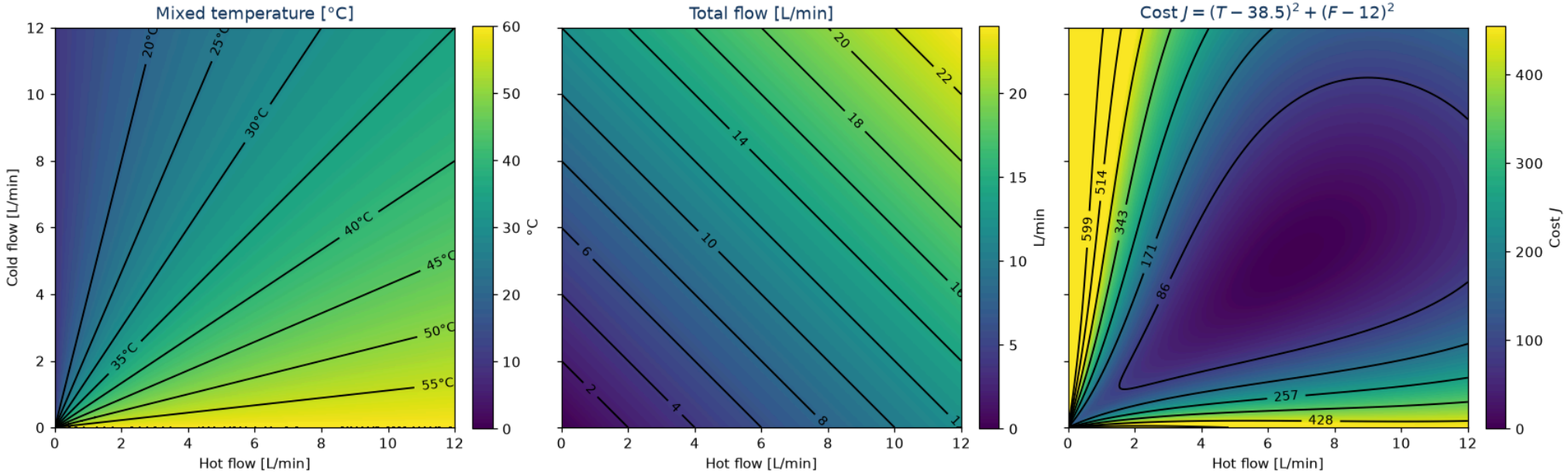
But the true curve is unknown — we can only probe it to search for the minimum.





Optimization example 2D: mixing hot and cold water

Mixing cold and hot water is an optimization task. We need to set a comfortable flow **and** a comfortable temperature.



Hot water 60 °C, cold 10 °C, target $T = 38.5$ °C at $F = 12$ L/min — two equations, two unknowns: $H + C = 12$, $\frac{60H+10C}{12} = 38.5 \Rightarrow H = 6.84,\ C = 5.16$ L/min, the dark spot in the right map.

Definition of optimization

Problem definition:

$$\mathbf{x}^\star \in \arg\min_{\mathbf{x}\in X} f(\mathbf{x})$$

Canonical form:

$$\begin{aligned} \text{minimize} \quad & f(\mathbf{x}) \\ \text{subject to} \quad & g_i(\mathbf{x}) \le 0, \quad i = 1, \dots, m \\ & h_j(\mathbf{x}) = 0, \quad j = 1, \dots, p \end{aligned}$$

$$X = \{\mathbf{x} \in \mathbb{R}^n : g_i(\mathbf{x}) \le 0,\ h_j(\mathbf{x}) = 0\}$$

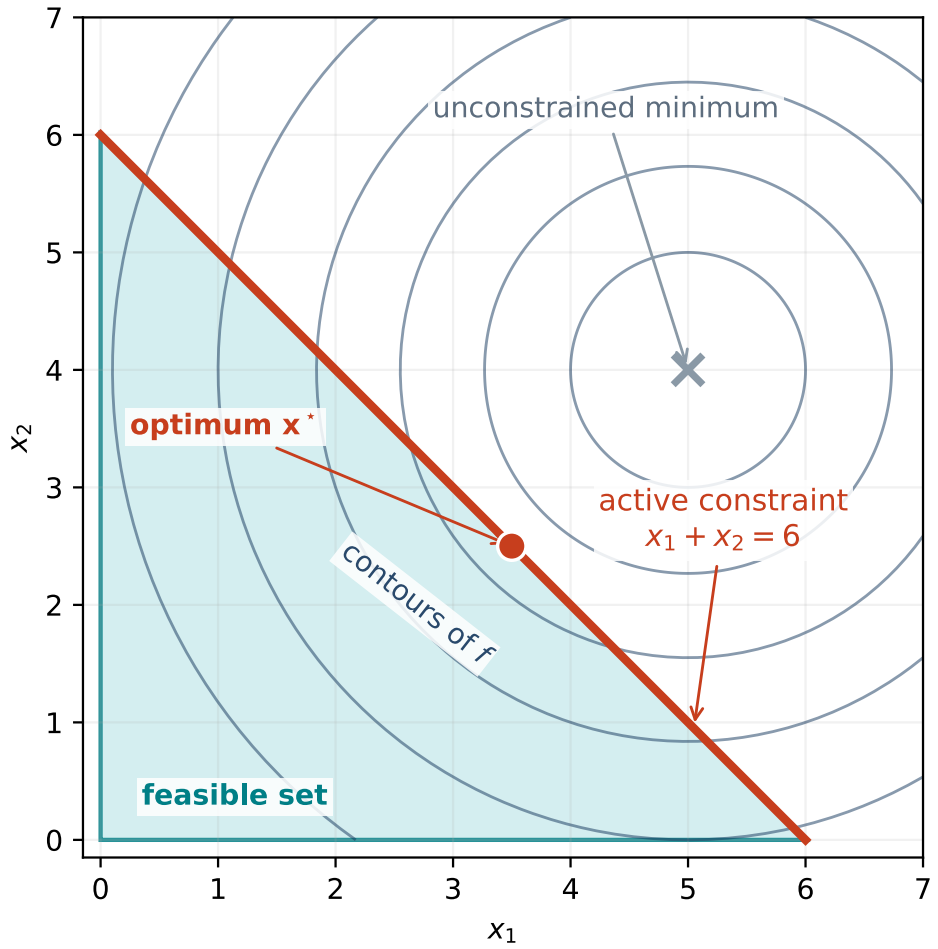
Every problem has the same three parts:

- **decision variables** $\mathbf{x}$ — what we may change (the speed, the two taps, the weights of a network);
- **objective** $f(\mathbf{x})$ — the single number we push down (cost, loss, error);
- **constraints** g_i, h_j — what must not be violated; together they carve out the **feasible set** X.

The minimum is not the minimizer. $p^\star = \min_{\mathbf{x}\in X} f(\mathbf{x})$ is a *number*, $\mathbf{x}^\star$ is the *point* at which it is attained — and one value may be attained at many points.

Convention: we always minimize. A maximization $\max_{\mathbf{x}} u(\mathbf{x})$ is solved as $\min_{\mathbf{x}} -u(\mathbf{x})$: the minimizer is the same, only the optimal value changes sign.

Feasible set and active constraints



- A point is **feasible** when it satisfies all the constraints at once.
- The **unconstrained** minimum may be infeasible — the optimum is then pushed onto the boundary.
- A constraint is **active** at $\mathbf{x}^\star$ when it holds with **equality** there. The active constraints are what stops the optimum from moving further.
- Here $x_1 + x_2 \leq 6$ is active, while $x_1 \geq 0$ and $x_2 \geq 0$ are not.

A generic two-variable example, not the tap — the geometry is easier to see on it.

Constrained optimization: a Gin & Tonic

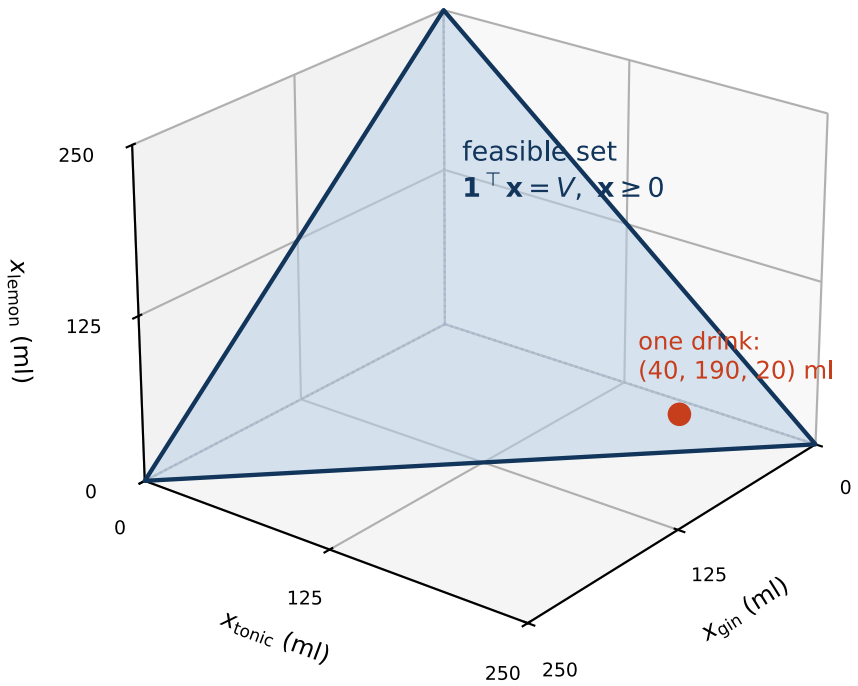
Task: mix a Gin & Tonic from gin, tonic water and lemon juice so that the **glass is full**. Decision variables (ml): $\mathbf{x} = (x_{\text{gin}}, x_{\text{tonic}}, x_{\text{lemon}})^\top \geq 0$.

Objective (unknown, possibly non-linear): maximize the taste score $T(\mathbf{x})$.

Simple constrained problem:

$$\begin{aligned} \text{minimize} \quad & -T(\mathbf{x}) \\ \text{subject to} \quad & \mathbf{1}^\top \mathbf{x} = V \quad \text{(glass is full)} \\ & \mathbf{x} \geq \mathbf{0} \end{aligned}$$

The feasible set is the triangle on the right: every drink that fills the glass ($V = 250$ ml) is one point of it.



$T(\mathbf{x})$ has no formula and no derivative — its value comes from one expensive evaluation, a tasting. Problems like that are called **black-box**; tuning the hyperparameters of a model is one of them.

Hard constraint or soft preference?

Constraint — “this must not be violated”

$$g_i(\mathbf{x}) \leq 0$$

A safety limit, an available budget, a law of physics. A hard constraint can make the problem **infeasible** — there may be no point satisfying it.

Objective term — “the less of it the better”

$$f(\mathbf{x}) = \text{error} + \lambda \cdot \text{penalty}$$

A preference weighed against the rest; λ says how much we care. A penalty can never make the problem infeasible, only shift the optimum.

The same requirement can often be modelled either way, and the two do not give the same answer. Ridge and lasso regression in the fourth lecture are exactly this move: the budget $\|\mathbf{w}\|_2^2 \leq t$ rewritten as the penalty $\lambda\|\mathbf{w}\|_2^2$ inside the objective.

The solver optimizes what we wrote down, not what we meant. Minimize the training error alone and the model memorizes the data; maximize sensitivity alone and the best test declares everybody ill. What is missing is always a term or a constraint that nobody wrote down.

Convexity in optimization: convex functions

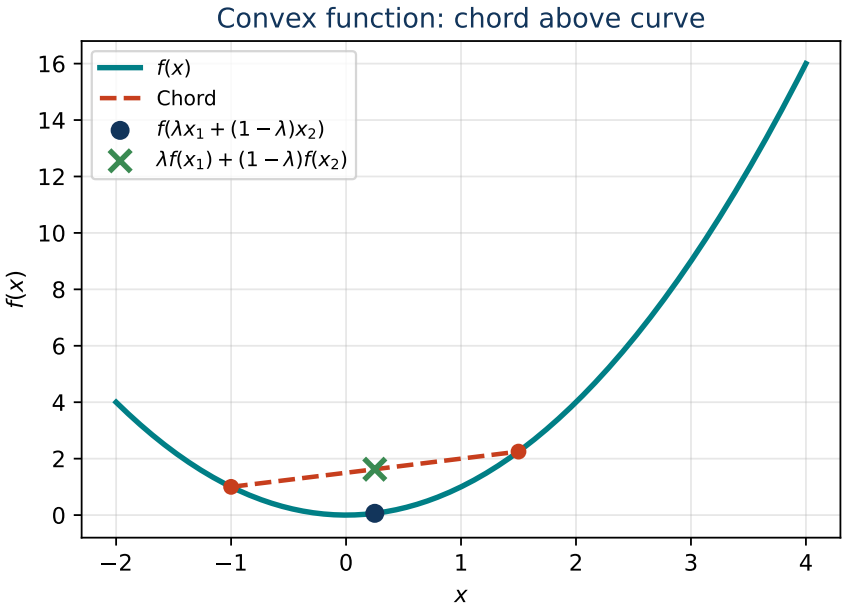
Convex functions

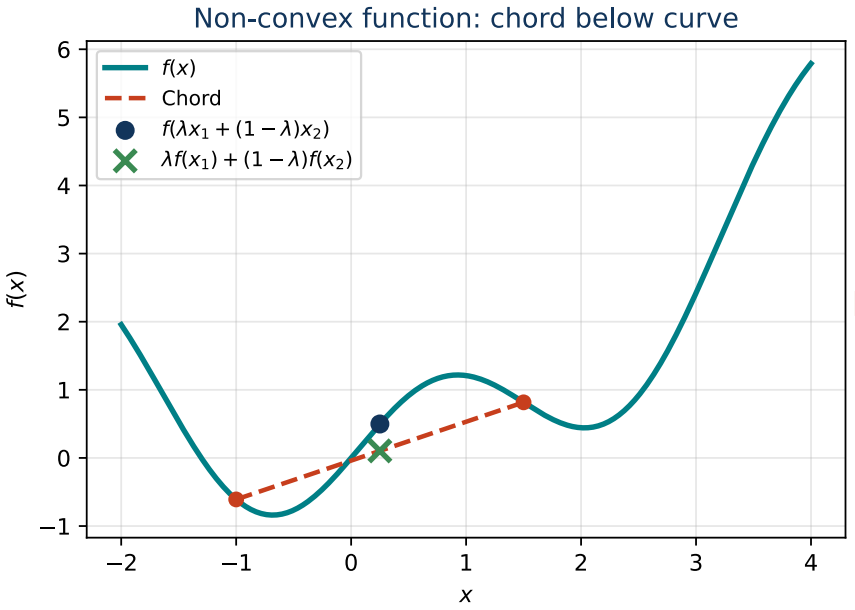
- A function $f : \mathbb{R}^n \to \mathbb{R}$ is convex if

$$f(\lambda \mathbf{x}_1 + (1-\lambda)\mathbf{x}_2) \;\leq\; \lambda f(\mathbf{x}_1) + (1-\lambda) f(\mathbf{x}_2)$$

  for all $\mathbf{x}_1, \mathbf{x}_2$ and $\lambda \in [0, 1]$.
- Intuition: the function lies **below the chord** connecting any two points on it.
- No local minima other than the global one — going downhill always reaches it.

Easy





Hard

Convexity in optimization: convex sets

Convex sets

- A set $C \subseteq \mathbb{R}^n$ is convex if

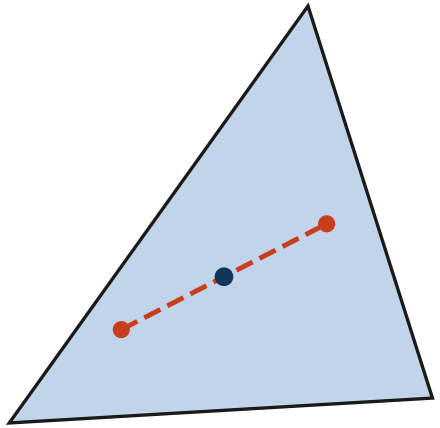
$$\lambda \mathbf{x}_1 + (1-\lambda)\mathbf{x}_2 \in C$$

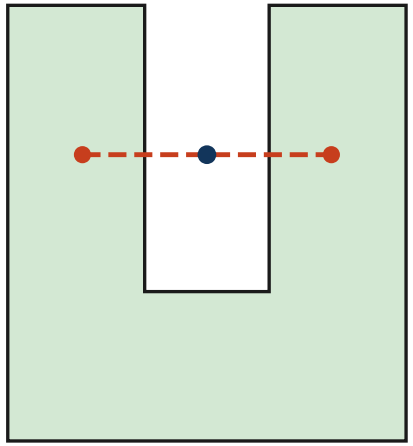
  for all $\mathbf{x}_1, \mathbf{x}_2 \in C$ and $\lambda \in [0,1]$.
- Intuition: the line segment between any two points in C lies in C.
- The minimum is not hidden behind a barrier — we can walk downhill to the best point.

Convex set (midpoint inside)

Non-convex set (midpoint outside)

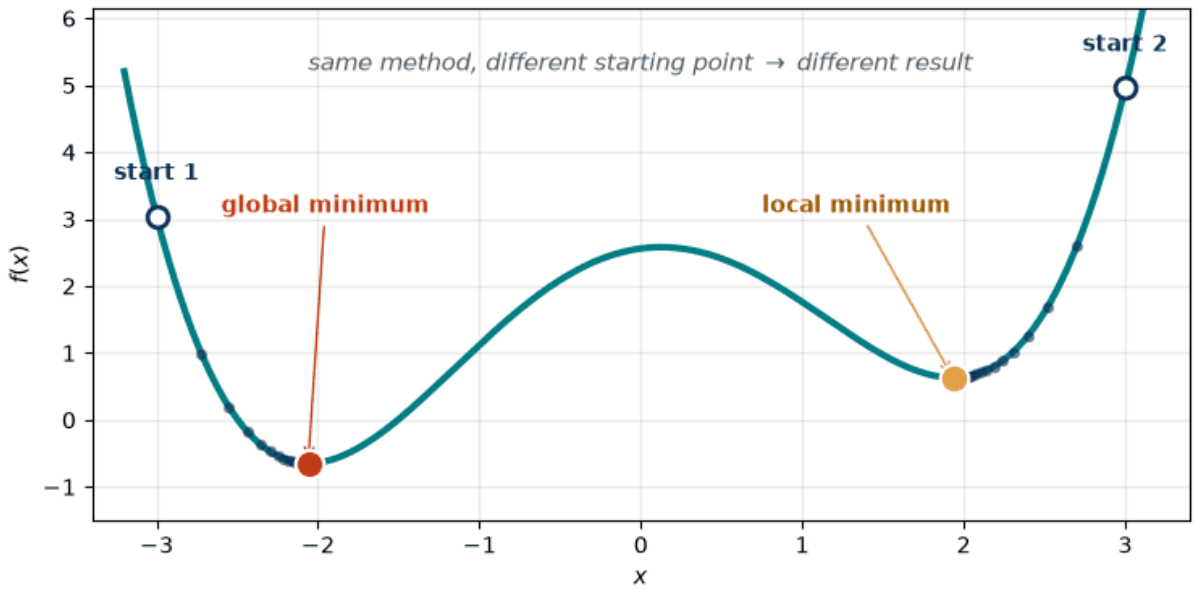
Easy





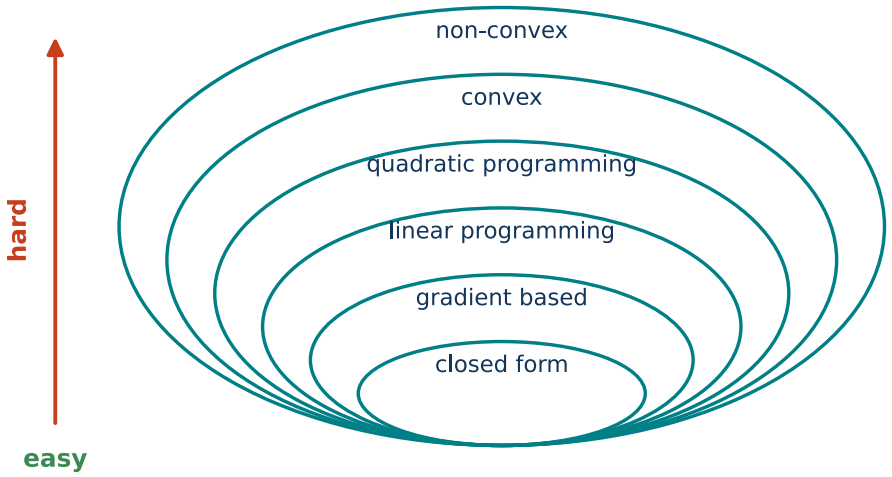
Hard

Local vs. global minimum — and what convexity buys



- **Local minimum:** the best point of some neighbourhood; **global minimum:** the best point of the whole feasible set.
- A local method ends in the valley its **starting point** leads it into — and cannot tell that a better one exists.
- **In a convex problem** — a convex objective over a convex feasible set — **every local minimum is a global one**. That is the whole reason convexity is worth talking about.
- It still does not guarantee that a minimizer **exists** ($\min_x e^x$ has none), that it is **unique**, or that the model is the right one.

Optimization approaches



- *Closed form (analytical solution)* — a special case; we get an equation that provides the solution without iterations.
- *Gradient-based* — if we can compute the derivative of $f(\mathbf{x})$, we can use it for faster optimization.
- *Linear programming* — linear $f(\mathbf{x})$ and constraints; standard fast solvers can be used.
- *Quadratic programming* — quadratic $f(\mathbf{x})$ and linear constraints; standard fast solvers can be used.
- *Convex* — typically easy to solve, one global minimum.
- *Non-convex* — typically hard to solve; we usually use a heuristic without guarantees.
- *Discrete optimization* — usually NP-hard, discrete domain.

Closed form and *gradient-based* name a **method**, the rest of the list names a **property of the problem**. It is the structure of the problem that narrows the choice of method — never the other way round.

Optimization approaches: examples

Almost everything (especially in ML) is optimization. We encounter it in many methods:

- *Closed form:* ridge regression (with matrix inverse), Gaussian fitting, PCA (with eigenvectors).
- *Gradient-based:* logistic regression, neural networks.
- *Linear programming:* L_1 regression (least absolute deviations regression).
- *Quadratic programming:* soft-margin SVM.
- *Convex:* lasso regression, logistic regression.
- *Non-convex:* deep neural networks, k-means clustering, GMM.
- *Discrete optimization:* feature selection.

Closed form solution of speed vs. fuel consumption

Simple model:

$$\mathrm{Fuel}(v) = \frac{A}{v} + Bv + C$$

Optimum condition (derivative equal to zero):

$$\frac{d}{dv}\mathrm{Fuel}(v) = 0 \;\Rightarrow\; -\frac{A}{v^2} + B = 0$$

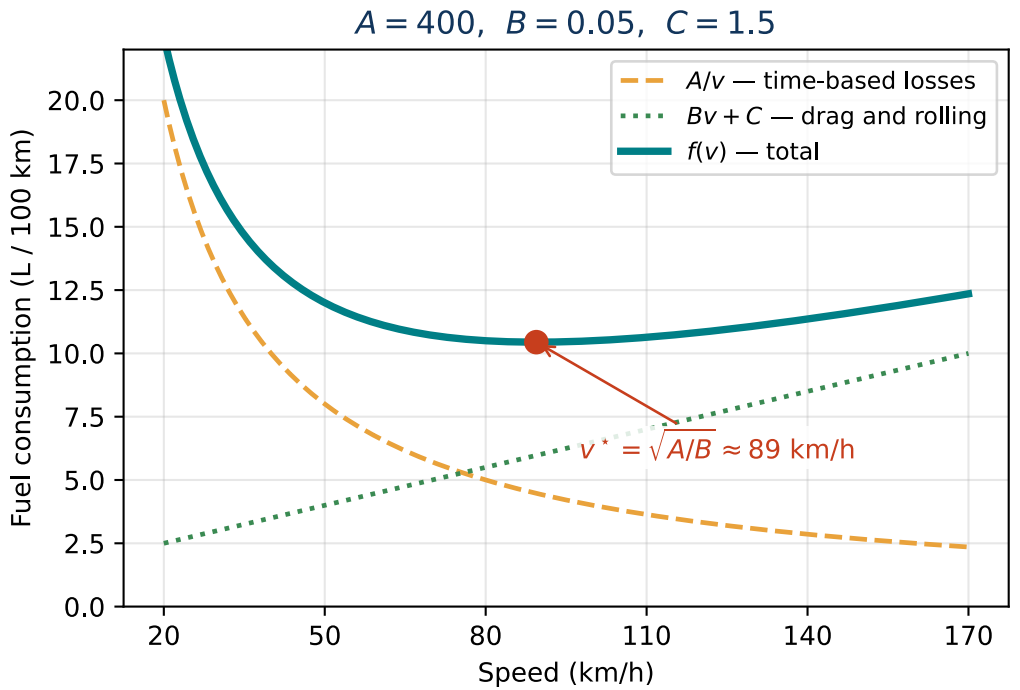
Solution:

$$v^\star = \sqrt{\frac{A}{B}}$$

Since the second derivative is positive ($\frac{2A}{v^3} > 0$), the optimum is a **unique minimum**.

With $A = 400, B = 0.05$: $v^\star = \sqrt{8000} \approx 89.4$ km/h and $\mathrm{Fuel}(v^\star) = 2\sqrt{AB} + C \approx 10.4$ L/100 km.

A — low-speed inefficiency, B — drag (kept linear in v), C — baseline consumption. With a limit $50 \le v \le 130$ km/h nothing changes: $v^\star$ lies inside, so the constraints are **inactive**.



Gradient descent

Intuition: if you change the input and see that the output (objective) is better, you continue in this direction. If it is worse, you start changing it in the opposite direction. (Think of the speed and the water temperature.)

The gradient points in the direction of the *steepest increase*; to minimize, we step **against** it.

The **gradient** $\nabla f(\mathbf{x})$ is a vector of partial derivatives:

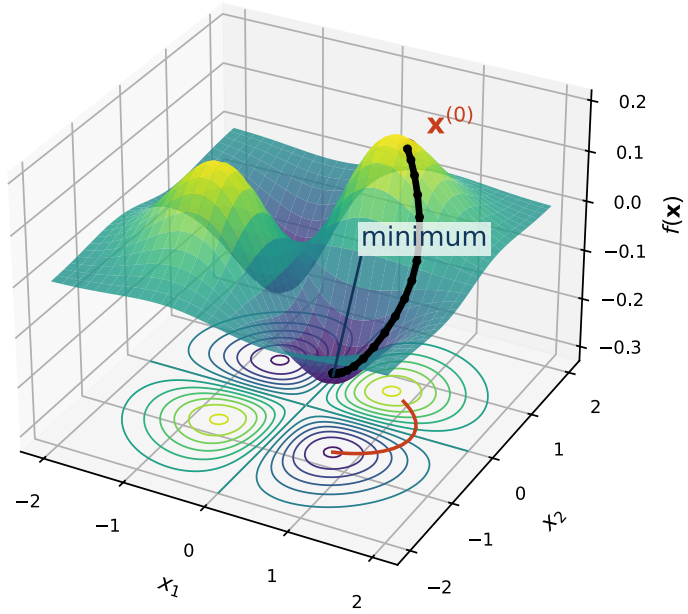
$$\nabla f(\mathbf{x}) = \left(\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \ldots, \frac{\partial f}{\partial x_n} \right)$$

Gradient descent update rule:

$$\mathbf{x}^{(t+1)} = \mathbf{x}^{(t)} - \eta \nabla f(\mathbf{x}^{(t)})$$

where $\eta > 0$ is the **learning rate** (step size).

Very important for ML and artificial neural networks.



Gradient descent step by step: the fuel example

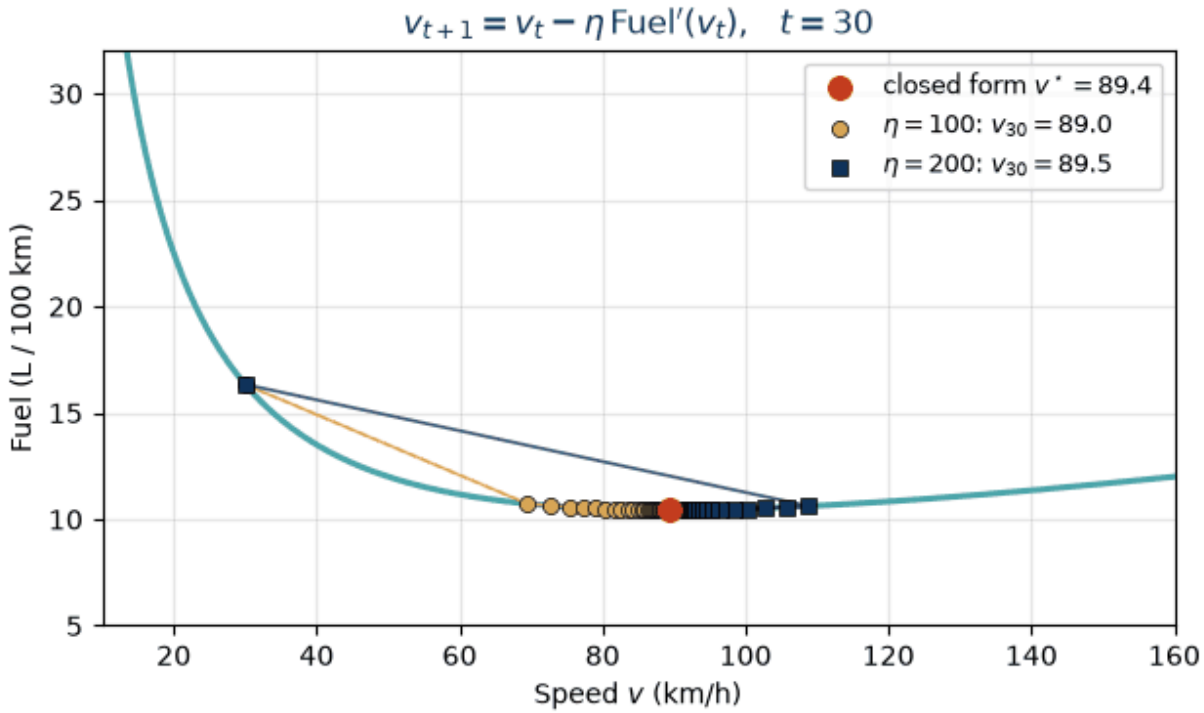
One variable, so the gradient is just the derivative:

$$\mathrm{Fuel}'(v) = -\frac{A}{v^2} + B, \qquad v_{t+1} = v_t - \eta\,\mathrm{Fuel}'(v_t)$$

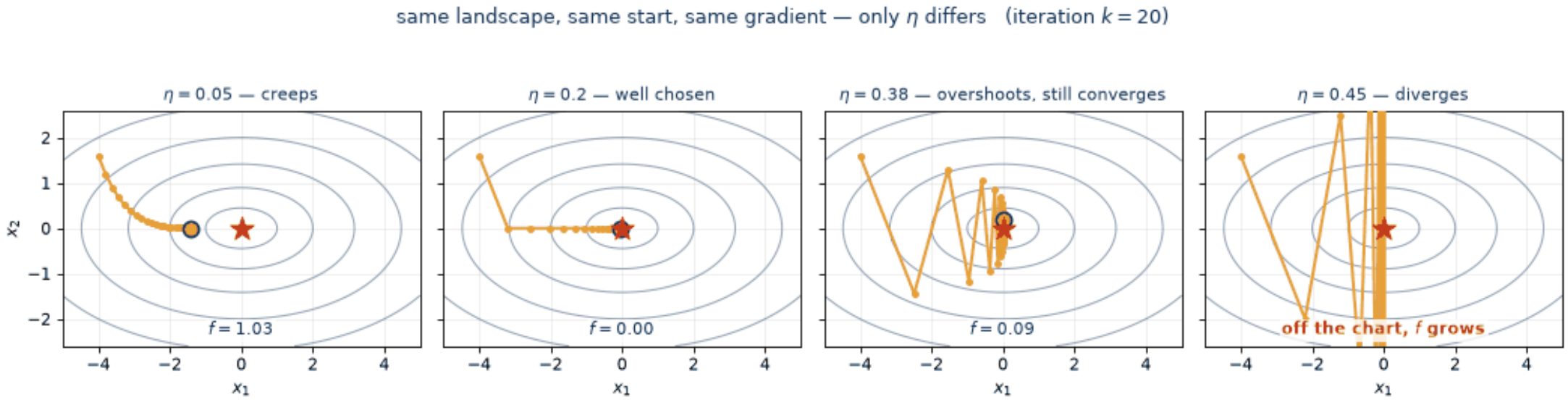
Start at $v_0 = 30$ km/h with $\eta = 200$:

t	v_t	$\mathrm{Fuel}'(v_t)$	v_{t+1}
0	30.0	$-400/900 + 0.05 = -0.394$	108.9
1	108.9	$+0.016$	105.6
2	105.6	$+0.014$	102.8
⋮			
30	89.5	≈ 0	

Both learning rates end at the closed-form optimum $v^\star = 89.4$ — but a large η overshoots the minimum in the first step, a small one creeps.



Gradient descent: the step size decides everything



On $f(\mathbf{x}) = \frac{1}{2}\left(x_1^2 + 5x_2^2\right)$ the update is $x_i \leftarrow (1 - \eta q_i)\, x_i$ with $q_1 = 1$, $q_2 = 5$: every iteration multiplies the error by $|1 - \eta q_i|$. Descent therefore converges **exactly when**

$$|1 - \eta q_i| < 1 \ \text{ for all } i \qquad \Longleftrightarrow \qquad \eta < \frac{2}{L},$$

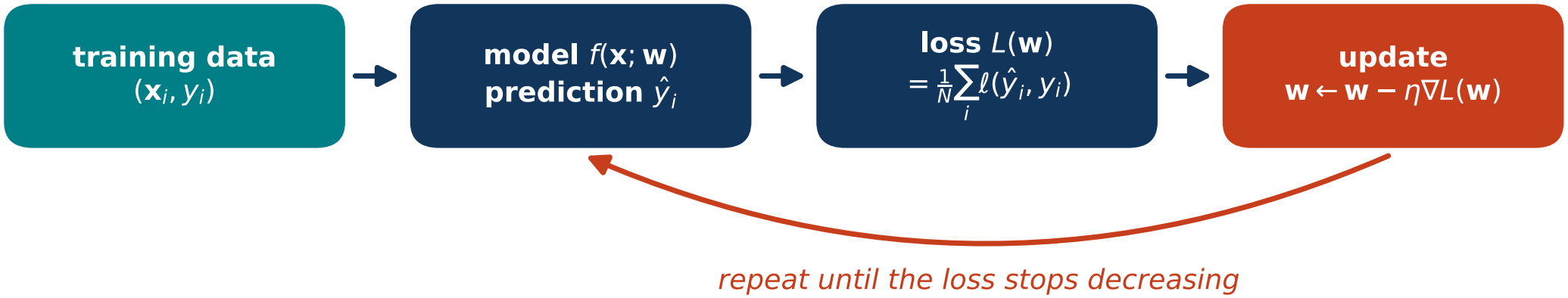
where $L = \max_i q_i = 5$ is the largest curvature — hence the limit $\eta < 0.4$ seen in the panels.

When to stop? A small gradient $\|\nabla f(\mathbf{x})\| < \varepsilon$, a small relative change of f or of $\mathbf{x}$, and always a cap on the number of iterations. When training a model, also the error on validation data (second lecture).

A small gradient only means “near a **stationary point**” — a maximum, a saddle or a flat plateau qualifies too. In a convex problem it really is the global minimum.

Training a model is optimization

training a model = minimizing $L(\mathbf{w})$ over the weights $\mathbf{w}$



- The **variables** are the parameters $\mathbf{w}$, not the data: a line has 2 of them, a small network 10^6, a large language model 10^{11}.
- The **objective** is the loss averaged over the training set.
- There are usually **no constraints**, which is why plain gradient descent is enough.

- No formula for $\mathbf{w}^\star$ exists — but we can evaluate L and ∇L at a point, and that is all descent needs.
- From the sixth lecture on, every neural network in this course is trained by exactly this loop; only f and the way the step is chosen change.
- Even k-means from the previous block is this: an objective, a bad landscape and a heuristic without guarantees.

Recap Quiz (1/2)

- **Classical programming vs. ML:** what are the inputs and outputs in each paradigm?
- **AI vs. ML vs. DL:** which is the umbrella term and how do ML and DL differ?
- **Branches:** give one task typical for *supervised*, *unsupervised*, *reinforcement* and *self-supervised* learning.
- **Regression vs. classification:** what is predicted in each case?
- **Distances:** write the formulas for $\|\mathbf{x}\|_2$, $\|\mathbf{x}\|_1$ and $\|\mathbf{x}\|_\infty$ (for $\mathbf{x} = \mathbf{a} - \mathbf{b}$).
- **1-NN vs. k-NN:** how does the prediction differ and what is k trading off?
- **k-means:** what objective does it minimize and what makes it sensitive?

Recap Quiz (2/2)

- **UPGMA:** how is the distance to a newly merged cluster computed?
- **Optimization tasks:** define an everyday task and an image or signal processing task as an optimization problem — name the variables, the objective and the constraints.
- **Feasible set:** when is a constraint *active* at the optimum, and what does that tell us?
- **Minimum vs. minimizer:** which of the two is a number and which one is a point?
- **Convexity:** name one property of convex functions/sets that simplifies optimization — and one thing convexity does *not* guarantee.
- **Gradient & GD:** define $\nabla f(\mathbf{x})$ and write one gradient descent update step.
- **GD:** is the gradient a scalar or a vector? How many dimensions does it have?
- **GD:** what happens when the learning rate is too large, and what sets the limit?
- **Analytical optimization:** how do you compute the minimum of a function (e.g. $f(x) = 3x^2 + 5x + 1$) analytically?